

১. DbContext Configuration

ডেটাবেস কানেকশন এবং DbContext রেজিস্টার করার জন্য এটি প্রয়োজন।

- কি কি লাগে: Connection String এবং কোন SQL Provider (যেমন: SQL Server) ব্যবহার করবেন তা উল্লেখ করতে হয়।

C#

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>  
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

`builder.Services.AddDbContext<ApplicationDbContext>( ... )` এই লাইনটি .NET-এ Dependency Injection (DI) বা সংক্ষেপে **DI Container**-এর সাথে আপনার ডেটাবেস ক্লাসটিকে পরিচয় করিয়ে দেয়। এর বিস্তারিত ব্যাখ্যা নিচে দেওয়া হলো:

Dependency Injection (DI) Container কী?

সহজ কথায়, DI Container হলো একটি "স্টোরেজ বক্স" বা "ম্যানেজার"। আপনি যখন অ্যাপ্লিকেশন রান করবেন, তখন এই বক্সের মধ্যে আপনার প্রয়োজনীয় সব ক্লাস বা সার্ভিস (যেমন: DbContext, Email Service) জমা করে রাখেন। পরবর্তীতে আপনার কন্ট্রোলার বা সার্ভিসে যখন ওই ক্লাসটির প্রয়োজন হয়, তখন আপনাকে নতুন করে `new ApplicationDbContext()` লিখতে হয় না; ফ্রেমওয়ার্ক নিজেই ওই বক্স থেকে অবজেক্টটি তৈরি করে আপনাকে দেয়।

এই লাইনটি কী করে?

এটি **Entity Framework Core** কে বলে দেয় যে:

- ApplicationDbContext নামে তোমার DbContext ক্লাসটি **Dependency Injection** এর মাধ্যমে অটোমেটিক ইনজেক্ট হবে।
- ডাটাবেস হিসেবে **SQL Server** ব্যবহার করবে।
- কানেকশন স্ট্রিং `appsettings.json` ফাইল থেকে নেবে।

প্রত্যেক অংশের বিস্তারিত ব্যাখ্যা:

অংশ	ব্যাখ্যা
<code>builder.Services</code>	<code>IServiceCollection</code> — এখানে সব সার্ভিস রেজিস্টার করা হয়
<code>AddDbContext<ApplicationDbContext></code>	<code>ApplicationDbContext</code> ক্লাসটিকে <code>DI Container</code> -এ রেজিস্টার করে
<code>(options => ...)</code>	ল্যাম্বডা এক্সপ্রেশন — <code>DbContext</code> কীভাবে কনফিগার করবেন তা বলার জন্য
<code>options.UseSqlServer(...)</code>	<code>SQL Server Provider</code> ব্যবহার করবে
<code>builder.Configuration.GetConnectionString("DefaultConnection")</code>	<code>appsettings.json</code> থেকে কানেকশন স্ট্রিং নিয়ে আসে

২. Identity System

ইউজার ম্যানেজমেন্ট এবং রোল এর জন্য Identity রেজিস্টার করা হয়।

- কি কি লাগে: User এবং Role ক্লাস এবং স্টোর হিসেবে DbContext-কে চিনিয়ে দিতে হয়।

C#

```
builder.Services.AddIdentity<ApplicationUser, IdentityRole>()  
    .AddEntityFrameworkStores<ApplicationDbContext>()  
    .AddDefaultTokenProviders();
```

এই কোডটি কী করে?

এটি **ASP.NET Core Identity** সিস্টেমকে অ্যাপ্লিকেশনে রেজিস্টার করে। এটি ইউজার ম্যানেজমেন্টের জন্য পুরো ফ্রেমওয়ার্ক প্রদান করে।

মূল কাজগুলো:

- ইউজার রেজিস্ট্রেশন
- লগইন / লগআউট
- পাসওয়ার্ড হ্যাশিং
- রোল ব্যবস্থাপনা (Admin, User, Moderator ইত্যাদি)
- Email Confirmation, Password Reset, Two-Factor Authentication
- Security Stamp, Lockout ইত্যাদি

প্রত্যেক অংশের বিস্তারিত ব্যাখ্যা:

অংশ	ব্যাখ্যা
AddIdentity<ApplicationUser, IdentityRole>	Identity সিস্টেম চালু করে। ApplicationUser = ইউজার টেবিলের ক্লাস, IdentityRole = রোল টেবিলের ক্লাস
.AddEntityFrameworkStores<ApplicationDbContext>()	বলে দেয় যে ইউজার ও রোলের ডাটা ApplicationDbContext এর মাধ্যমে SQL Database-এ সংরক্ষণ হবে

.AddDefaultTokenProviders()

Password Reset, Email
Confirmation, Phone Confirmation
ইত্যাদির জন্য টোকেন জেনারেট করার
সুবিধা দেয়

সম্পূর্ণ **Recommended** ভাৰ্সন (Best Practice)

C#

```
builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    // Password Policy
    options.Password.RequireDigit = true;
    options.Password.RequireLowercase = true;
    options.Password.RequireUppercase = true;
    options.Password.RequireNonAlphanumeric = true;
    options.Password.RequiredLength = 8;

    // Lockout Policy
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(5);
    options.Lockout.MaxFailedAccessAttempts = 5;
    options.Lockout.AllowedForNewUsers = true;

    // User Settings
    options.User.RequireUniqueEmail = true;
    options.SignIn.RequireConfirmedEmail = true;    // ইমেইল কনফার্ম না করে লগইন করতে দেবে না
    options.SignIn.RequireConfirmedPhoneNumber = false;
})
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddDefaultTokenProviders();
```

ApplicationUser ক্লাস কেমন হবে?

C#

```
public class ApplicationUser : IdentityUser<Guid> // Guid ব্যবহার করা ভালো
{
    public string FullName { get; set; } = string.Empty;
    public string? RefreshToken { get; set; }
    public DateTime RefreshTokenExpiryTime { get; set; }
    // আরও যা যা দরকার (Address, ProfilePicture ইত্যাদি)
}
```

DbContext এ যোগ করতে হবে:

C#

```
public class ApplicationDbContext : IdentityDbContext<ApplicationUser, IdentityRole<Guid>, Guid>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { }
}
```

৩. JWT (Authentication)

এপিআই সিকিউরিটির জন্য JWT ব্যরিয়্যার টোকেন কনফিগার করতে হয়।

- কি কি লাগে: Issuer, Audience এবং Secret Key।

C#

```
builder.Services.AddAuthentication(options => {
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
}).AddJwtBearer(options => {
    options.TokenValidationParameters = new TokenValidationParameters {
        ValidateIssuer = true,
        ValidateAudience = true,
        IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes("YourSecretKey"))
    };
});
```

এই কোডটি কী করে?

এটি **JWT Bearer Token Authentication** সেটআপ করে। এর মাধ্যমে তোমার API শুধুমাত্র বৈধ JWT টোকেনযুক্ত রিকোয়েস্ট গ্রহণ করবে। এটি API সিকিউরিটির সবচেয়ে জনপ্রিয় ওয়ে।

প্রত্যেক লাইনের বিস্তারিত ব্যাখ্যা:

কোড	ব্যাখ্যা
AddAuthentication()	Authentication মিডলওয়্যার চালু করে
DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme	ডিফল্ট স্কিম হিসেবে JWT ব্যবহার করবে

AddJwtBearer()	JWT Bearer টোকেন হ্যান্ডলার যোগ করে
TokenValidationParameters	টোকেন যাচাই করার নিয়মসমূহ সেট করে
ValidateIssuer = true	টোকেনে Issuer থাকা বাধ্যতামূলক
ValidateAudience = true	টোকেনে Audience থাকা বাধ্যতামূলক
IssuerSigningKey	টোকেন সাইন করার জন্য Secret Key

Recommended / Best Practice ভাঙ্গন (2025-2026)

C#

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true, // টোকেনের মেয়াদ শেষ হয়ে গেলে Reject করবে
            ValidateIssuerSigningKey = true, // Key যাচাই করবে
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!)),

            ClockSkew = TimeSpan.Zero // টোকেন expire হওয়ার সাথে সাথে Reject
        };

        // Optional: Authorization header না পেলে কী মেসেজ দেখাবে
        options.Events = new JwtBearerEvents
        {
            OnAuthenticationFailed = context =>
            {
                context.Response.StatusCode = 401;
                return Task.CompletedTask;
            }
        };
    });
```

appsettings.json এর জন্য সেটিংস

JSON

```
{
  "Jwt": {
    "Issuer": "https://yourdomain.com",
    "Audience": "https://yourdomain.com",
    "Key": "YourSuperSecretKeyAtLeast32CharactersLong1234567890",
    "TokenExpiryInMinutes": 60,
    "RefreshTokenExpiryInDays": 7
  }
}
```

 গুরুত্বপূর্ণ: Key কমপক্ষে **32** ক্যারেক্টার এর হতে হবে। প্রোডাকশনে অবশ্যই User Secrets / Azure Key Vault ব্যবহার করবে।

Identity + JWT একসাথে ব্যবহার করলে (সবচেয়ে কমন)

C#

```
// 1. Identity
builder.Services.AddIdentity<ApplicationUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();

// 2. JWT Authentication (এটি Identity এর পরে আসবে)
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(...); // উপরের কোড
```

Extension Method আকারে (Best Practice)

C#

```
public static class JwtExtensions
{
    public static IServiceCollection AddJwtAuthentication(this IServiceCollection services, IConfiguration configuration)
    {
        services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
            .AddJwtBearer(options =>
            {
                options.TokenValidationParameters = new TokenValidationParameters
                {
                    ValidateIssuer = true,
                    ValidateAudience = true,
```

```

        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = configuration["Jwt:Issuer"],
        ValidAudience = configuration["Jwt:Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(configuration["Jwt:Key"]!)),
        ClockSkew = TimeSpan.Zero
    };
});

return services;
}
}

```

Program.cs এ ব্যবহার:

```

C#
builder.Services.AddJwtAuthentication(builder.Configuration);

```

অতিরিক্ত গুরুত্বপূর্ণ সেটিংস (যা প্রায়ই লাগে)

- RequireExpirationTime = true
- ValidateActor = false
- SaveToken = true (কন্ট্রোলারে HttpContext.GetTokenAsync() ব্যবহার করতে চাইলে)

চাইলে আমি পুরো **JWT Token Generate Service, Login Response, Refresh Token** সহ সম্পূর্ণ সেটআপও দেখিয়ে দিতে পারি।

8. CQRS & MediatR

পুরো অ্যাপ্লিকেশনের কমান্ড এবং কুয়েরি হ্যান্ডলারগুলো রেজিস্টার করার জন্য এটি ব্যবহার হয়।

- কি কি লাগে: কোন অ্যাসেম্বলিতে হ্যান্ডলারগুলো আছে তা বলে দিতে হয়।

```

C#
builder.Services.AddMediatR(cfg =>
    cfg.RegisterServicesFromAssembly(typeof(Program).Assembly));

```

এই লাইনটি কী করে?

MediatR হলো একটি **Mediator Pattern** এর লাইব্রেরি। এটি CQRS (Command Query Responsibility Segregation) প্যাটার্ন বাস্তবায়ন করতে সাহায্য করে।

এই কোডটি **MediatR** কে বলে দেয়:

- তোমার অ্যাপ্লিকেশনে কোন কোন **Command, Query, Handler, Notification, Validator** আছে।
- সবগুলোকে অটোমেটিকভাবে **Dependency Injection** এ রেজিস্টার করে দাও।

অর্থাৎ, তুমি যদি `CreateUserCommand` লিখো এবং `CreateUserCommandHandler` লিখো, তাহলে এই লাইনটি না দিলে `Handler` কাজ করবে না।

বিস্তারিত ব্যাখ্যা

অংশ	ব্যাখ্যা
<code>AddMediatR</code>	MediatR সার্ভিস কনফিগারেশন শুরু করে
<code>cfg =></code>	Configuration অবজেক্ট (<code>MediatRServiceConfiguration</code>)
<code>RegisterServicesFromAssembly</code>	নির্দিষ্ট Assembly থেকে সব MediatR ক্লাস (Handler, Request, Notification) খুঁজে রেজিস্টার করে
<code>typeof(Program).Assembly</code>	বর্তমান প্রজেক্টের (Web API প্রজেক্ট) Assembly

বড় প্রজেক্টে **Best Practice (Recommended)**

বড় অ্যাপ্লিকেশনে শুধু `Program Assembly` রেজিস্টার করলে চলে না। সাধারণত **Clean Architecture** এ নিচের মতো করা হয়:

C#

```
builder.Services.AddMediatR(cfg =>
{
    cfg.RegisterServicesFromAssemblies(
        typeof(Application.AssemblyReference).Assembly, // Application Layer
        typeof(Program).Assembly, // Web/API Layer (যদি দরকার হয়)
        typeof(Infrastructure.AssemblyReference).Assembly // যদি কোনো Handler থাকে
    );
});
```

AssemblyReference ক্লাস বানানো ভালো প্র্যাকটিস:

Application/AssemblyReference.cs

C#

```
public static class AssemblyReference
{
    public static readonly Assembly Assembly = typeof(AssemblyReference).Assembly;
}
```

পুরো **Recommended** কনফিগারেশন (2025 Best Practice)

C#

```
builder.Services.AddMediatR(cfg =>
{
    cfg.RegisterServicesFromAssemblies(
        typeof(Application.AssemblyReference).Assembly);

    // Optional but Recommended
    cfg.NotificationPublisherType = typeof(NotificationPublisher); // যদি Notification ব্যবহার করো
});

// Pipeline Behaviors (খুবই গুরুত্বপূর্ণ)
builder.Services.AddTransient(typeof(IPipelineBehavior<,>), typeof(ValidationBehavior<,>));
builder.Services.AddTransient(typeof(IPipelineBehavior<,>), typeof(LoggingBehavior<,>));
builder.Services.AddTransient(typeof(IPipelineBehavior<,>), typeof(PerformanceBehavior<,>));
```

কী কী অটো রেজিস্টার হয়?

- IRequest<T> → Command & Query
 - IRequestHandler<TRequest, TResponse>
 - INotification ও INotificationHandler
 - IValidator<T> (FluentValidation এর সাথে)
 - IPipelineBehavior<,>
-

Extension Method আকারে (Best Practice)

C#

```
public static class MediatRExtensions
{
    public static IServiceCollection AddMediatRConfiguration(this IServiceCollection services)
    {
        services.AddMediatR(cfg =>
        {
            cfg.RegisterServicesFromAssemblies(
                typeof(Application.AssemblyReference).Assembly);
        });
    }
}
```

```
// Pipeline Behaviors
services.AddTransient(typeof(IPipelineBehavior<,>), typeof(ValidationBehavior<,>));
services.AddTransient(typeof(IPipelineBehavior<,>), typeof(UnhandledExceptionBehavior<,>));
services.AddTransient(typeof(IPipelineBehavior<,>), typeof(LoggingBehavior<,>));

return services;
}
}
```

Program.cs এ ব্যবহার:

```
C#
builder.Services.AddMediatRConfiguration();
```

অতিরিক্ত টিপস

1. **FluentValidation** যোগ করলে:
C#
 2. `services.AddValidatorsFromAssembly(typeof(Application.AssemblyReference).Assembly);`
 3. **Multiple Assemblies** রেজিস্টার করা যায়।
 4. **Performance** এর জন্য শুধু Application Layer এর Assembly রেজিস্টার করাই যথেষ্ট।
-

চাইলে আমি পরবর্তীতে এগুলো দেখাতে পারি:

- ValidationBehavior এর পুরো কোড
- একটা Sample Command + Handler + Validator
- LoggingBehavior

৫. Swagger Configuration

API ডকুমেন্টেশনে JWT লক আইকন (Authorization) যোগ করার জন্য এটি একটু বিস্তারিতভাবে লিখতে হয়।

Security Definition (JWT Lock আইকন যোগ করা)

আপনার API যদি সিকিউরড হয় (যেমন: [Authorize] ব্যবহার করেন), তবে Swagger-এ একটি **"Authorize"** বাটন এবং তালার আইকন যোগ করতে হয়। এটি ছাড়া আপনি Swagger থেকে টোকেন পাঠিয়ে API টেস্ট করতে পারবেন না।

- কি কি ইনফরমেশন লাগে:

- **Scheme:** সাধারণত "Bearer" ব্যবহার হয়।
- **Type:** এটি "Http" বা "ApiKey" হতে পারে (JWT এর জন্য Http)।
- **In:** টোকেনটি কোথায় থাকবে? (Header-এ)।

C#

```
builder.Services.AddSwaggerGen(c => {
    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme {
        In = ParameterLocation.Header,
        Description = "Please enter token",
        Name = "Authorization",
        Type = SecuritySchemeType.Http,
        BearerFormat = "JWT",
        Scheme = "bearer"
    });
});
```

এই কোডটি কী করে?

এটি **Swagger UI**-তে **JWT Bearer Token** এর জন্য একটি **Authorize** বাটন তৈরি করে।

যখন তুমি Swagger UI ওপেন করবে (/swagger), তখন উপরের ডানদিকে একটা  **Authorize** বাটন দেখতে পাবে। সেখানে তুমি Bearer <your-jwt-token> পেস্ট করে সব protected API টেস্ট করতে পারবে।

প্রত্যেক প্রপারটির বিস্তারিত ব্যাখ্যা:

প্রপারটি	মান	ব্যাখ্যা
AddSecurityDefinition("Bearer", ...)	"Bearer"	সিকিউরিটি স্কিমের নাম (যেকোনো নাম দিতে পারো)
In = ParameterLocation.Header	Header	টোকেন কোথায় পাঠানো হবে (সাধারণত Authorization Header)
Name = "Authorization"	Authorization	Header এর নাম
Type = SecuritySchemeType.Http	Http	HTTP এর জন্য (JWT এর জন্য সবচেয়ে ভালো)
Scheme = "bearer"	bearer	স্কিম হিসেবে bearer ব্যবহার

BearerFormat = "JWT"	JWT	Swagger কে বলে দেয় এটা JWT টোকেন
Description	...	ইউজারকে দেখানো টেক্সট

৬. Custom Services (Dependency Injection)

আপনার তৈরি করা ইন্টারফেস এবং ক্লাসগুলো রেজিস্টার করার জন্য।

- **Scope:** AddScoped, AddTransient, বা AddSingleton হিসেবে রেজিস্টার করতে হয়।

C#

```
builder.Services.AddScoped<IProductService, ProductService>();
```

৭. Email Service

মেইল পাঠানোর জন্য আলাদা কনফিগারেশন। সাধারণত SMTP সেটিংস এর মাধ্যমে করা হয়।

C#

```
builder.Services.AddTransient<IEmailSender, EmailSender>();
```

৮. Controller & JSON Response

এপিআই কন্ট্রোলার এবং রেসপন্স ফরম্যাট ঠিক করার জন্য।

C#

```
builder.Services.AddControllers()
    .AddJsonOptions(options => {
        options.JsonSerializerOptions.PropertyNamingPolicy = null; // Original casing রাখতে চাইলে
    });
```

প্রো-টিপ: **Extension Method** ব্যবহার করে নোট নিন

ভবিষ্যতে ভুল এড়াতে Program.cs-এ সব না লিখে ServiceExtensions নামে একটি ক্লাস করতে পারেন:

C#

```
public static class ServiceExtensions
{
    public static void ConfigureJwt(this IServiceCollection services, IConfiguration configuration)
    {
```

```

        // JWT এর কোড এখানে থাকবে
    }

    public static void ConfigureServices(this IServiceCollection services, IConfiguration
configuration)
    {
        // DB এর কোড এখানে থাকবে
    }
}

```

এরপর Program.cs-এ শুধু এভাবে কল করবেন:

```

C#
builder.Services.ConfigureDbContext(builder.Configuration);
builder.Services.ConfigureJwt(builder.Configuration);

```

এভাবে গুছিয়ে রাখলে কোড রিডেবল থাকবে এবং ভবিষ্যতে বড় প্রজেক্টে কাজ করতে সুবিধা হবে।

✅ ভালো প্রশ্ন!

আমরা যেগুলো আলোচনা করেছি (DbContext, Identity, JWT, MediatR, Swagger, Custom Services) ছাড়াও বেশ কিছু গুরুত্বপূর্ণ সার্ভিস আছে যেগুলো বেশিরভাগ প্রজেক্টে Program.cs এ রেজিস্টার করতে হয়।

নিচে সবচেয়ে কমন এবং প্রয়োজনীয় সার্ভিসগুলো ক্যাটাগরি অনুসারে দিলাম:

1. Basic & Must Have Services

```

C#
builder.Services.AddControllers();           // Controllers
builder.Services.AddEndpointsApiExplorer();   // Swagger এর জন্য
builder.Services.AddSwaggerGen(...);         // Swagger

// CORS (Frontend থেকে API কল করতে চাইলে)
builder.Services.AddCors(options =>
{
    options.AddDefaultPolicy(policy =>
    {
        policy.AllowAnyOrigin()
            .AllowAnyHeader()
            .AllowAnyMethod();
        // অথবা নির্দিষ্ট Origin: .WithOrigins("https://yourfrontend.com")
    });
});

```

2. খুবই কমন সার্ভিস (প্রায় সব প্রজেক্টে লাগে)

C#

// AutoMapper

builder.Services.AddAutoMapper(typeof(Program).Assembly);

builder.Services.AddAutoMapper(typeof(Application.AssemblyReference).Assembly);

// FluentValidation

builder.Services.AddValidatorsFromAssembly(typeof(Application.AssemblyReference).Assembly);

// HttpClient Factory

builder.Services.AddHttpClient();

// Options Pattern

builder.Services.Configure<JwtSettings>(builder.Configuration.GetSection("Jwt"));

builder.Services.Configure<EmailSettings>(builder.Configuration.GetSection("EmailSettings"));

// Response Compression

builder.Services.AddResponseCompression();

3. অন্যান্য গুরুত্বপূর্ণ সার্ভিস

সার্ভিস	কোড	কখন লাগে
Health Check	builder.Services.AddHealthChecks();	প্রোডাকশনে মনিটরিং এর জন্য
Rate Limiting	builder.Services.AddRateLimiter(...);	API Abuse রোধ করতে
Memory Cache	builder.Services.AddMemoryCache();	সাধারণ ক্যাশিং
Distributed Cache (Redis)	builder.Services.AddStackExchangeRedisCache(...);	বড় প্রজেক্টে
Hangfire (Background Job)	builder.Services.AddHangfire(...);	Background jobs চালাতে

SignalR	<code>builder.Services.AddSignalR();</code>	Real-time ফিচারের জন্য
Hosted Service	<code>builder.Services.AddHostedService<MyBackgroundService>();</code>	Background Task

4. Program.cs এ সাজানো সম্পূর্ণ উদাহরণ (Recommended Order)

C#

```
var builder = WebApplication.CreateBuilder(args);

// ===== Basic =====
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();

// ===== Database & Identity =====
builder.Services.AddApplicationDbContext(builder.Configuration);
builder.Services.AddIdentityConfiguration();

// ===== Authentication =====
builder.Services.AddJwtAuthentication(builder.Configuration);

// ===== Application Layer =====
builder.Services.AddMediatRConfiguration();
builder.Services.AddAutoMapper(typeof(Application.AssemblyReference).Assembly);
builder.Services.AddValidatorsFromAssembly(typeof(Application.AssemblyReference).Assembly);

// ===== Custom Services =====
builder.Services.AddApplicationServices(); // তোমার সব IxxxService
builder.Services.AddEmailService(builder.Configuration);

// ===== Swagger =====
builder.Services.AddSwaggerConfiguration();

// ===== Cross Cutting =====
builder.Services.AddCors();
builder.Services.AddHttpClient();
builder.Services.AddMemoryCache();

// ===== Authorization =====
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", p => p.RequireRole("Admin"));
});
```

তোমার প্রজেক্টে এখন কী কী লাগতে পারে?

আমাকে বলো তোমার প্রজেক্টের ধরন:

- সাধারণ **CRUD API** → উপরের মধ্যে Basic + CORS + AutoMapper যথেষ্ট
- বড় এন্টারপ্রাইজ প্রজেক্ট → Rate Limiting, Redis, Hangfire, Health Check
- **Real-time** ফিচার → SignalR
- ফাইল আপলোড + ইমেইল → Email + File Upload Service

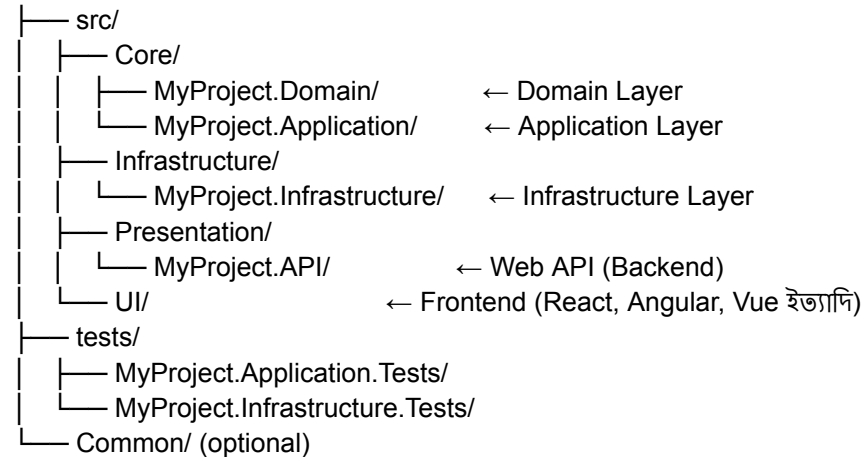
বলো, তোমার প্রজেক্টে এখনো কোন সার্ভিস যোগ করতে চাও বা কোনটা নিয়ে বিস্তারিত জানতে চাও? আমি সেই অনুযায়ী Extension Method সহ কোড দিয়ে দিব।

=====

1. Solution Structure (Recommended)

text

MyProject.sln



Layer-wise বিস্তারিত বর্ণনা

1. Domain Layer (MyProject.Domain)

এখানে ব্যবসায়িক নিয়ম থাকবে। কোনো Framework/Dependency নেই।

- **Entities**

- **Value Objects**
- **Enums**
- **Domain Events**
- **Exceptions** (Domain-specific)

উদাহরণ:

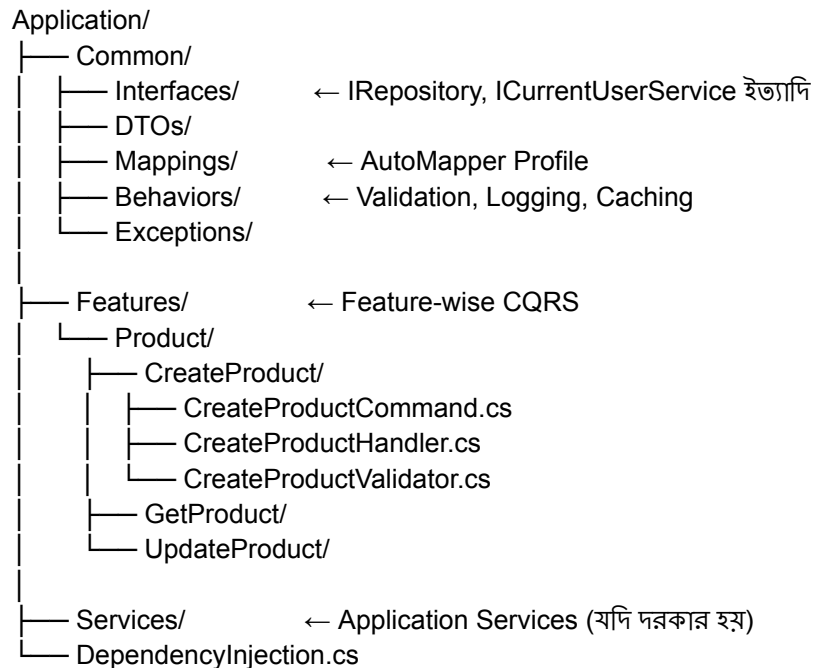
- User.cs, Product.cs, Order.cs
- Money.cs (Value Object)
- OrderCreatedEvent.cs

2. Application Layer (MyProject.Application)

এটি **Use Cases** এর জায়গা।

ফোল্ডার স্ট্রাকচার:

text



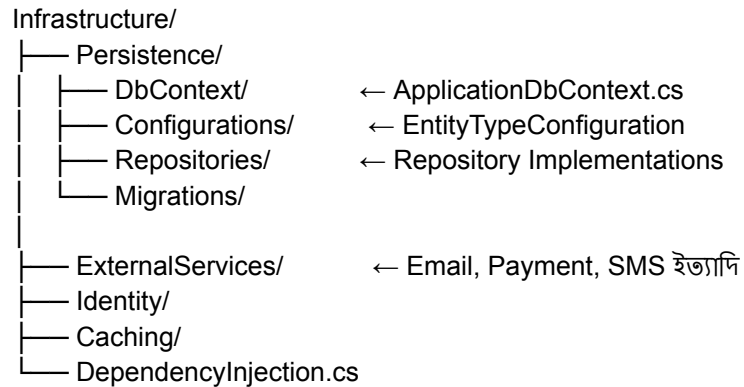
ব্যবহৃত টুলস:

- **MediatR** → Command/Query Handler
- **FluentValidation**
- **AutoMapper**

3. Infrastructure Layer (MyProject.Infrastructure)

ফোল্ডার স্ট্রাকচার:

text



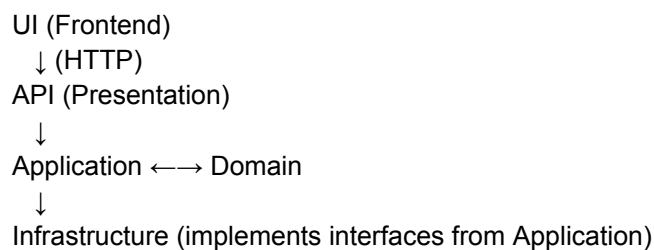
4. Presentation Layer (MyProject.API)

- Controllers/
- Program.cs
- appsettings.json
- Middleware/
- Filters/

Frontend আলাদা প্রজেক্ট হিসেবে রাখবে (React/Vue/Angular)। API শুধু JSON দিয়ে communicate করবে।

Dependency Flow (খুব গুরুত্বপূর্ণ)

text



Dependency Rule: Outer Layer → Inner Layer (শুধুমাত্র inward dependency)

NuGet Packages (মূল)

Application:

- MediatR
- FluentValidation
- AutoMapper
- MediatR.Extensions.Microsoft.DependencyInjection

Infrastructure:

- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools
- Microsoft.AspNetCore.Identity.EntityFrameworkCore (যদি Identity ব্যবহার করো)

API:

- Swashbuckle.AspNetCore (Swagger)
- Serilog